Informe del Trabajo Obligatorio

Sistema de Gestión del Torneo Interfacultades de Fútbol 5

Materia: Base de Datos 1
Integrantes: Lucia Rodríguez, Bruno Rodríguez

Introducción

El objetivo de este trabajo obligatorio fue desarrollar un sistema para gestionar el Torneo Interfacultades de Fútbol 5 de la Universidad Católica del Uruguay.

Buscamos facilitar la organización del torneo mediante el registro de equipos, jugadores, partidos, resultados, goles, tarjetas y sanciones. También permite consultar información del torneo y generar reportes a partir de los datos almacenados.

Utilizamos MySQL como gestión de base de datos y Python para crear una aplicación que permita interactuar con la información.

Durante el desarrollo tuvimos que realizar cambios en la estructura inicial, corregir errores y agregar validaciones para cumplir con las reglas establecidas en la consigna.

Desarrollo del proyecto

Primera versión de la base de datos

Al comenzar el trabajo, tomamos como referencia las tablas propuestas en la letra del obligatorio e intentamos construir la base de datos siguiendo esa estructura.

En esta primera etapa fuimos creando las tablas, definiendo sus atributos y estableciendo las relaciones mediante claves primarias y foráneas.

A medida que avanzamos, identificamos algunas dificultades relacionadas con la organización de las tablas y la implementación de determinadas funcionalidades.

Además, realizamos distintas modificaciones sobre la estructura y los datos utilizando sentencias SQL. Con el tiempo, estos cambios comenzaron a generar confusión, porque resultaba difícil identificar qué modificaciones correspondían a la versión definitiva de la base de datos.

Segunda versión de la base de datos

Debido a las dificultades encontradas, decidimos comenzar una nueva versión del proyecto desde cero.

Esta decisión no significó descartar todo el trabajo anterior. Aprovechamos algunas de las tablas, relaciones e ideas que ya habíamos desarrollado, pero reorganizamos la base de datos para trabajar sobre una estructura más clara.

El principal objetivo fue evitar gran cantidad de modificaciones agrupadas y contar con una versión más ordenada, que pudiéramos entender y mantener durante el resto del desarrollo.

A partir de ese momento, separamos mejor la creación de las tablas, la carga de los datos de prueba y la programación de las funcionalidades.

Consideramos que esta decisión fue importante, porque nos permitió continuar con una base más organizada y entender mejor las relaciones entre las distintas tablas.

Construcción de la base de datos definitiva

En la nueva versión desarrollamos las tablas necesarias para representar la información del torneo, incluyendo:

- Usuarios y facultades.
- Jugadores, equipos y sus planteles.
- Torneos y sus distintas ediciones.
- Complejos deportivos, canchas y turnos.
- Partidos y planillas de juego.
- Goles, tarjetas y sanciones.

Utilizamos claves primarias para identificar los registros y claves foráneas para relacionar las tablas y mantener la integridad referencial.

También incorporamos datos de prueba, incluyendo 36 jugadores, seis equipos, tres jornadas disputadas (nueve partidos inicialmente), goles, tarjetas y sanciones. Registramos también un autogol para comprobar que se contabilizara a favor del equipo contrario. En las pruebas posteriores agregamos un décimo partido programado, que utilizamos para comprobar la reprogramación.

Estos datos permiten comprobar el funcionamiento de las consultas y representar situaciones que pueden ocurrir durante un torneo real.

La primera estructura quedó distribuida en 14 tablas. Posteriormente añadimos fecha_torneo e historial_partido, alcanzando 16 tablas. fecha_torneo registra las jornadas de cada edición de manera independiente de los partidos. La clave foránea compuesta (id_torneo, numero_fecha) en partido impide asignar un encuentro a una jornada inexistente. historial_partido mantiene la programación anterior y nueva cuando se modifica un encuentro, junto con el motivo y la fecha del cambio. , el partido mantiene la programación vigente sin perder la trazabilidad de las reprogramaciones.

La creación, carga y evolución del esquema se organizaron en archivos separados: 01_creacion.sql (estructura inicial), 02_datos.sql (datos de prueba), 03_mejoras.sql (jornadas, historial y relaciones) y 03_updates.sql (actualización de credenciales de prueba). Las consultas de informes se almacenaron en 04_reportes.sql, y los permisos de usuarios MySQL se documentaron en 05_permisos.sql.

Decisiones de implementación

MySQL

Utilizamos MySQL para crear y administrar la base de datos, de acuerdo con los requisitos del obligatorio.

Durante el desarrollo utilizamos DataGrip para ejecutar sentencias SQL, consultar las tablas y verificar los datos almacenados.

La utilización de una base de datos relacional nos permitió organizar la información en distintas entidades y establecer relaciones entre ellas.

Python

Desarrollamos la aplicación utilizando Python y la biblioteca mysql.connector, que permite ejecutar consultas SQL desde el programa.

Decidimos implementar una aplicación de consola mediante menús, porque nos resultó una alternativa sencilla y adecuada para concentrarnos en las funcionalidades principales del obligatorio.

No utilizamos un ORM, las operaciones con la base de datos se realizan mediante sentencias SQL.

También dividimos el código en diferentes archivos según su funcionalidad, como jugadores, equipos, partidos, planillas, goles, tarjetas, sanciones, usuarios y reportes.

Esta separación nos ayudó a mantener el código más organizado y a identificar con mayor facilidad dónde realizar cada modificación.

Para los reportes, funciones/reportes.py lee las consultas guardadas en 04_reportes.sql y las ejecuta usando la misma función de conexión que el resto de la aplicación. Un menú ofrece 15 opciones; el archivo SQL también conserva una consulta intermedia de resultados utilizada durante la construcción de la tabla de posiciones. Esto evita duplicar todas las consultas entre Python y el script SQL. La conexión se abre con mysql.connector, se recuperan los resultados mediante el cursor y se presentan en la consola.

Registro de información y validaciones

Implementamos operaciones de alta, baja y modificación para diferentes entidades del sistema.

Además, incorporamos validaciones relacionadas con las reglas del torneo. Entre ellas:

- Evitar que un equipo juegue contra sí mismo.
- Evitar enfrentamientos repetidos dentro de una misma edición.
- Impedir que una cancha sea asignada a dos partidos en el mismo día y turno.
- Comprobar que los equipos tengan suficientes jugadores para disputar un partido.
- Controlar que los jugadores pertenezcan a la facultad correspondiente.
- Limitar la cantidad de integrantes de los planteles.
- Impedir modificaciones de planteles fuera del período de inscripción.
- Evitar que jugadores suspendidos figuren en las planillas.
- Verificar que los goles y las tarjetas correspondan a jugadores registrados en la planilla del partido.
- Impedir la eliminación de partidos jugados.
- Incorporar un control de tiempo para las correcciones de resultados.

También implementamos el registro de sanciones automáticas a partir de tarjetas rojas y acumulación de tarjetas amarillas. Para estas operaciones se utilizan transacciones y, cuando se detecta un error de MySQL, se revierte la operación con ROLLBACK para evitar guardados parciales.

Reforzamos las validaciones de partidos y planteles después de probar la aplicación. El módulo de partidos consulta la disponibilidad de equipos, cancha y turno, exige que el número de jornada exista y restringe cambios de estado. En una reprogramación, actualiza partido y agrega el registro correspondiente en historial_partido dentro de la misma transacción. El módulo de equipos verifica el estado inscripción antes de solicitar los datos de un nuevo integrante, controla la correspondencia de facultad, los números de camiseta, la cantidad máxima de integrantes y el límite conjunto de docentes y funcionarios.

En planillas, goles y tarjetas incorporamos verificaciones del estado del partido y del plazo de 48 horas calculado a partir de su fecha y turno de finalización. Los goles y tarjetas solamente pueden registrarse para jugadores presentes en la planilla. En el caso de las planillas, también se evita quitar a un jugador si tiene goles o tarjetas registrados en el encuentro. Las sanciones disciplinarias se gestionan por separado de las automáticas y se comprueba que el jugador pertenezca a la edición correspondiente.

Estas validaciones fueron agregándose progresivamente a medida que avanzamos en la implementación y en las pruebas.

Usuarios y permisos

El sistema prevee dos roles principales: administrador y planillero.

El administrador dispone de un menú para gestionar los diferentes elementos del torneo. El planillero cuenta con opciones relacionadas con el registro de planillas, goles y tarjetas.

También agregamos una opción de consulta pública para visualizar partidos y resultados sin necesidad de iniciar sesión.

Esta última funcionalidad fue una mejora que decidimos incorporar para facilitar el acceso a la información del torneo.

Para reforzar la separación de funciones, creamos usuarios específicos en MySQL: admin_torneo, planillero_torneo y consulta_torneo. Estos no son los mismos registros que las cuentas de la tabla login: los primeros determinan los privilegios de conexión a la base de datos y las segundas determinan qué menú puede utilizar una persona en la aplicación.

admin_torneo dispone de permisos de consulta e inserción, actualización y eliminación sobre las tablas del sistema. planillero_torneo tiene permisos de lectura sobre las tablas necesarias para operar, sin permiso de lectura sobre login, y permisos de escritura limitados a planillas, goles, tarjetas y creación de sanciones. consulta_torneo permite autenticar a los usuarios y realizar las consultas públicas de partidos y resultados. La selección del usuario de MySQL se realiza en database.py según el rol activo; antes de iniciar sesión se utiliza el usuario de consulta. Las contraseñas MySQL se solicitan mediante getpass y se conservan solamente en memoria durante la ejecución, sin incluirlas en el código fuente.

Durante las pruebas detectamos que los registros de la tabla login contenían contraseñas de prueba sin transformar, mientras Python comparaba valores hash SHA-256. Corregimos los datos y mantuvimos un script de actualización para reproducir la configuración inicial. Reconocemos, sin embargo, que SHA-256 sin sal no es un mecanismo adecuado para producción: una evolución del sistema debería migrar a un algoritmo específico para contraseñas, como Argon2id o bcrypt. Asimismo, el usuario consulta_torneo conserva permiso de lectura sobre la tabla login para la autenticación actual, lo cual implica acceso a los hashes; esa exposición debe restringirse con un diseño de autenticación más seguro. No consideramos esta limitación resuelta por el solo hecho de separar los roles.

Mejoras implementadas y consideradas

Durante el desarrollo realizamos algunas mejoras respecto de la primera versión del proyecto.

Una de las principales fue reorganizar la base de datos y separar los scripts SQL, lo que permitió distinguir la creación de la estructura de la inserción de datos.

Otra mejora fue dividir el código Python en módulos, evitando concentrar todas las funcionalidades en un único archivo.

También agregamos mensajes más claros al mostrar jugadores, equipos y partidos. Inicialmente, algunas consultas mostraban los registros como tuplas de Python, lo que dificultaba su lectura. Posteriormente, modificamos la presentación de los datos para que fueran más comprensibles.

Incorporamos además una consulta pública del torneo y de los resultados.

Entre las mejoras del modelo destacamos la tabla fecha_torneo, que permite registrar jornadas antes de programar sus encuentros, y historial_partido, que conserva las modificaciones realizadas a la programación. También implementamos la actualización del historial y del partido en una misma transacción, de manera que no se guarde una reprogramación incompleta.

Elaboramos los reportes requeridos sobre tabla de posiciones, goleadores, fair play, promedios de goles por fecha y edición, uso de canchas y turnos, jugadores por facultad y tipo, jugadores sin participaciones, sanciones por equipo y motivo, y porcentajes por estado de partido. Como consultas adicionales incorporamos resultados individuales de encuentros, equipos invictos, cantidad de partidos disputados por equipo y jugadores con más tarjetas. En total, se implementaron 15 opciones de reporte en el menú, utilizando JOIN, agregaciones, CASE, GROUP BY, subconsultas, NOT EXISTS, UNION ALL y expresiones comunes de tabla (WITH) cuando fue necesario. La tabla de posiciones se deriva de los goles registrados y contempla expresamente los autogoles, sin mantener un marcador duplicado en otra tabla.

Como posibles mejoras futuras, consideramos la implementación de una interfaz gráfica, un sistema de autenticación más avanzado y controles adicionales de integridad y seguridad.

Pruebas realizadas

Durante el desarrollo realizamos pruebas desde la aplicación Python y desde DataGrip.

Entre las pruebas se encuentran:

- Visualización de equipos, jugadores y partidos.
- Consulta de planteles y planillas.
- Cálculo de resultados a partir de los goles registrados.
- Comprobación del funcionamiento de los autogoles.
- Intentos de eliminar partidos que ya habían sido jugados.
- Intentos de registrar jugadores en equipos que no participan en un partido.
- Validación de jugadores suspendidos.
- Intentos de registrar goles y tarjetas a jugadores no incluidos en la planilla.
- Comprobación de restricciones de inscripción de jugadores en equipos.

Estas pruebas nos permitieron detectar errores, corregir funciones y comprobar distintas restricciones del sistema.

En la última etapa se realizaron las siguientes verificaciones concretas:

Prueba	Resultado observado
Consultar jugadores y equipos desde Python	La aplicación recuperó los registros de MySQL. Inicialmente se mostraban como tuplas y se mejoró la presentación en varias pantallas.
Consultar los 15 reportes desde el menú de Python	Se ejecutaron las 15 opciones y se obtuvieron resultados, según las pruebas locales realizadas.
Relacionar partidos con fecha_torneo	La consulta de comprobación devolvió correctamente los nueve partidos originales asociados a las jornadas 1, 2 y 3.
Intentar modificar un partido jugado	El programa rechazó la modificación.
Crear y reprogramar un encuentro	Se creó el partido 10; al modificar su programación, se registró la nueva ubicación y se verificó el historial correspondiente en DataGrip.
Intentar inscribir un jugador cuando el torneo está en curso	La aplicación rechazó la operación antes de pedir la cédula.
Registrar goles en un encuentro antiguo o programado	La aplicación rechazó ambas operaciones por plazo o estado.
Registrar tarjetas en un encuentro antiguo o programado	La aplicación rechazó ambas operaciones por plazo o estado.
Ingresar una sanción para un jugador inexistente en la edición	Se rechazó la operación.
Iniciar sesión como administrador y planillero	Ambos accesos funcionaron tras corregir los hashes de prueba y configurar las cuentas específicas de MySQL.
Consultar los privilegios del planillero	Se inspeccionaron los permisos con SHOW GRANTS y se retiró el SELECT general para otorgarlo por tabla.

Las comprobaciones se realizaron en el entorno local de desarrollo.

Dificultades encontradas

La mayor dificultad que encontramos fue la organización inicial de la base de datos.

Al principio trabajamos directamente sobre las tablas de la consigna y fuimos introduciendo cambios a medida que aparecían nuevas necesidades. Esto nos generó confusión sobre las modificaciones realizadas y sobre cómo se relacionaban algunas partes del código.

Por ese motivo decidimos comenzar nuevamente, reutilizando parte de la estructura anterior.

También encontramos dificultades al conectar Python con MySQL, al organizar las funciones en archivos diferentes y al implementar algunas reglas del torneo, especialmente aquellas que dependían de información de varias tablas.

Otro aspecto que necesitó atención fue la validación de los datos antes de ejecutar operaciones como agregar jugadores, programar partidos y registrar sanciones.

Fuimos resolviendo estos problemas mediante pruebas, revisión del código y modificaciones progresivas.

Otra dificultad concreta fue distinguir las contraseñas de acceso al torneo de las contraseñas utilizadas por los usuarios de MySQL. Al principio, el programa dependía de una variable de entorno MYSQL_PASSWORD y devolvía un error si no estaba definida. Posteriormente configuramos la solicitud interactiva de las claves mediante getpass y separamos las conexiones según el rol. También debimos corregir la representación de las contraseñas de prueba en la tabla login, ya que no coincidía con el hash que buscaba la aplicación.

Para trabajar en equipo utilizamos GitHub. Primero se integraron los scripts de creación y datos de prueba, luego las mejoras y reportes, y después las funciones de Python y las validaciones mediante ramas, commits y solicitudes de incorporación (*pull requests*). Esto ayudó a reducir el riesgo de sobrescribir modificaciones del otro integrante. También aprendimos la diferencia entre descargar un ZIP del repositorio y clonarlo con Git para poder actualizarlo mediante git pull.

Conclusión

El desarrollo de este obligatorio nos permitió aplicar conceptos de Base de Datos 1 en un sistema que representa una situación concreta que pasa en la vida real.

Trabajamos con la creación de tablas, claves primarias y foráneas, relaciones entre entidades, consultas SQL y operaciones de inserción, modificación y eliminación.

También aplicamos conectar una base de datos MySQL con una aplicación desarrollada en Python y implementar reglas de negocio que dependen de varias tablas.

Creemos que uno de los aprendizajes más importantes fue comprender la importancia de planificar correctamente la estructura de la base de datos antes de avanzar demasiado con la programación.

La decisión de comenzar nuevamente el proyecto, aprovechando parte del trabajo inicial, nos permitió mejorar la organización y continuar el desarrollo de una manera más clara.

El agregado de jornadas e historial de reprogramaciones mostró que el modelo propuesto inicialmente podía ampliarse para representar mejor las necesidades de gestión del torneo. Los reportes, a su vez, permitieron utilizar los datos para obtener estadísticas sin almacenar resultados derivados de manera redundante.

La experiencia también reforzó la importancia de probar tanto casos válidos como inválidos, documentar las decisiones tomadas y distinguir las validaciones de Python de las restricciones y permisos de MySQL. Aunque se implementaron y probaron numerosas funcionalidades, reconocemos que quedan posibles mejoras de seguridad, pruebas integrales y automatización de controles para una versión destinada a un entorno real.

Bitácora del proyecto

La siguiente bitácora está organizada por etapas, porque no contamos con un registro de fechas.

Etapa	Actividades realizadas	Dificultades y decisiones
1. Análisis de la consigna	Identificación de entidades, relaciones, restricciones y funcionalidades.	Traducir las reglas deportivas a un modelo relacional.
2. Primera versión de la base	Creación inicial de tablas con claves primarias y foráneas.	Acumulación de cambios y dificultad para identificar la versión definitiva.
3. Reinicio y organización	Reconstrucción del esquema y separación de creación, datos y mejoras en scripts SQL.	Reutilizar lo válido de la primera versión sin conservar errores de organización.
4. Carga de datos	Facultades, 36 jugadores, seis equipos, planteles, complejos, canchas, turnos, nueve partidos iniciales, goles, tarjetas y sanciones.	Orden de inserción y coherencia de facultades, equipos y jugadores.
5. Integración con Python	Conexión mediante mysql.connector y organización de funciones en módulos.	Configuración de credenciales y errores de conexión.

6. Gestión del torneo	ABM de entidades, partidos, planillas, goles, tarjetas y sanciones.	Validaciones de integrantes, equipos participantes y eventos.
7. Mejoras del esquema	Incorporación de fecha_torneo e historial_partido, más una relación compuesta con partido.	Representar jornadas y mantener trazabilidad sin rehacer la base.
8. Reportes SQL	Desarrollo de reportes obligatorios y adicionales; integración de 15 opciones en Python.	Cálculo de autogoles, posiciones, agrupaciones y consultas extensas.
9. Reprogramaciones y pruebas	Creación de un décimo partido, modificación de programación y verificación del historial.	Evitar cambios a partidos jugados y validar conflictos de cancha y horarios.
10. Validaciones complementarias	Controles de período de inscripción, planteles, planillas, goles, tarjetas, sanciones y ventana de 48 horas.	Rechazar entradas inválidas antes de alterar registros.
11. Inicio de sesión y permisos	Separación de administrador, planillero y consultas públicas; creación de usuarios MySQL con privilegios diferentes.	Diferenciar roles de la aplicación y usuarios de MySQL; corregir hashes de prueba.
12. Integración colaborativa	Uso de ramas de Git, commits, pull requests, merges y actualizaciones desde GitHub.	Evitar sobrescribir el trabajo de otro integrante y mantener actualizada la copia local.
13. Preparación de entrega	Organización de scripts, documentación y revisión de resultados observados.	Queda por verificar la instalación completa desde cero y cerrar limitaciones conocidas de seguridad.

Bibliografía y recursos consultados

- Consigna del Trabajo Obligatorio de Base de Datos 1, Universidad Católica del Uruguay.
- Diapositivas, ejercicios y apuntes utilizados durante el curso de Base de Datos 1.
- ChatGPT, utilizado como herramienta para aclarar conceptos, revisar relaciones y examinar validaciones.